Streaming Incremental Decentralized Responsible Architectures

Ilaria Battiston
CWI
The Netherlands
ilaria@cwi.nl

Dandan Yuan
CWI
The Netherlands
dandan.yuan@cwi.nl

Peter Boncz
CWI
The Netherlands
boncz@cwi.nl

ABSTRACT

We present SIDRA, a local-first architecture where central warehouses only store aggregated data, with the raw base data residing in Personal Data Stores (PDSs). Its aim is to achieve *data minimization* by having individual users own and manage their data, yet enabling third-party analytics in a more privacy-friendly manner, using *Materialized Views* (MVs) over the union of these personal stores. We describe SIDRA SQL, which allows for the declarative specification of these decentralized architectures. The steady stream of new data generated by individual users necessitates Incremental View Maintenance (IVM), which we enhance with a trusted staging area that collects updates and flushes these to the central views only when considered privacy-friendly. We implement SIDRA as a compiler leveraging OpenIVM, a system that performs portable and cross-system IVM using pure SQL. We provide a threat model, and we show that SIDRA is compatible with incremental Differential Privacy (DP), obtaining additional security guarantees. Our evaluation of SIDRA using [todo]

PVLDB Reference Format:
Ilaria Battiston, Dandan Yuan, and Peter Boncz. SIDRA. PVLDB, 14(1): XXX-XXX, 2020.
doi:XX.XX/XXX.XX

PVLDB Artifact Availability:
The source code, data, and/or other artifacts have been made available at URL_TO_YOUR_ARTIFACTS.

1 INTRODUCTION

[todo: mention our column keywords here, edit the abstract and contributions] There is an ongoing privacy crisis where centralized information systems have shifted power away from individuals towards cloud-based application providers. This now dominant architecture manages user data in centralized databases, enabling data-driven decision-making through data science and machine learning, but raises concerns about privacy, security, and cost. The need to transmit all collected data to a single location and party takes away control from individuals and creates a risk of data leakage and abuse. Also, the centralization of all base data leads to significant cloud storage, network, and CPU costs.

A proposed strategy to mitigate these issues is *data minimization* [45], limiting the collection of personal information to what is strictly necessary for the application. This concept, fundamental to privacy regulations like the GDPR, aims to strike a balance between enabling centralized analyses and preserving user privacy. However, in practice, the existing interpretation of data minimization is coarse-grained, thereby still providing legal ground to centrally collect detailed user data, even if that data is used only in aggregated form — by lack of a decentralized place in the centralized architecture to process that base data. Decentralized data architectures are more complex than centralized ones, and enterprise data architects are often constrained by standard tools that lack the infrastructure to decentralize data. As a result, organizations continue to fully centralize (at least) all user data involved in application functions, perpetuating the very issues data minimization is meant to solve. This leads us to our first main research questions:

RQ1	Can we simplify the construction of analytics infrastructures storing base data decentrally with application users?
RQ1.1	Can these be specified from a declarative framework?
RQ1.2	How can we still centrally analyze decentralized user data while maintaining data minimization?
RQ1.3	How can end-users trust such privacy properties?

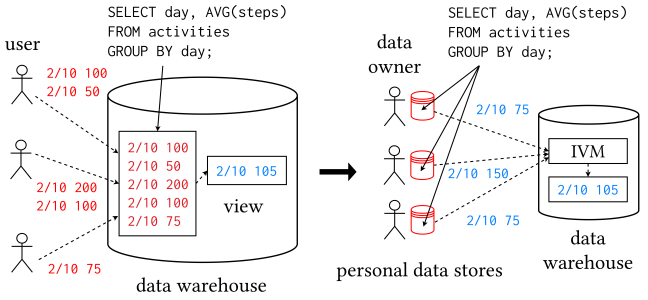

Figure 1: In a centralized architecture (left), raw sensitive user data (red) is collected and aggregated centrally (dashed line). In SIDRA (right), each PDS computes and shares incremental aggregates (blue). This design empowers users to maintain control over their data, minimizing server resource overhead while still providing centralized analytical capabilities.

We, therefore, propose SIDRA, a declarative SQL framework for specifying decentralized architectures where only analytical results over materialized views — rather than raw data — get published from Personal Data Stores (PDSs) to centralized analytics systems, as shown in Figure 1. This allows for a third party to analyze and base business decisions, while preserving the locality of sensitive data and retaining user control. We aim to make it practical for data architects to leave base data with individuals in PDSs, thereby removing the "need" to centralize.[1]

[1]Note that while regulations like GDPR can influence technology, technology could also influence legal interpretation. Adoption of SIDRA could have legal consequences; central storage of base data losing "purpose binding", and potentially becoming illegal.

This work is licensed under the Creative Commons BY-NC-ND 4.0 International License. Visit https://creativecommons.org/licenses/by-nc-nd/4.0/ to view a copy of this license. For any use beyond those covered by this license, obtain permission by emailing info@vldb.org. Copyright is held by the owner/author(s). Publication rights licensed to the VLDB Endowment.
Proceedings of the VLDB Endowment, Vol. 14, No. 1 ISSN 2150-8097.
doi:XX.XX/XXX.XX

These data architectures are "responsible", in the sense that they reduce the amount of data that gets materialized in centrally-hosted views and provide a toolbox for applying declarative privacy-preserving constraints — such as enforcing a minimum amount of contributions before publishing data, or attaching Differential Privacy (DP) functions directly in SQL.

Contributions. The main contributions of this paper are not in security or privacy techniques (or proofs), but in database architecture: we present SIDRA as a proposal to break free from centralized applications. We define the syntax and semantics of SIDRA SQL, allowing declarative specification of decentralized architectures with basic privacy properties. This contribution addresses RQ1. We analyze SIDRA's privacy model and discuss how DP can be integrated with our architecture, providing a framework complementary to incremental DP algorithms and decentralized processing, thereby addressing RQ1.3. Further, we create an open-source implementation that advances IVM by integrating and orchestrating decentralized processing with mechanisms to enforce basic privacy constraints. To address RQ2, we provide an evaluation of the full system running a combination of [todo] under a heavy workload, that show that cloud costs can be significantly reduced using SIDRA. Lastly, SIDRA opens a roadmap to follow-on security and privacy research, as discussed in §7.

2 THE SIDRA CONCEPT

SIDRA echoes the "local-first" movement[2]: applications can always use the full power of SQL on their personal data store, which can reside e. g., on a mobile device, enabling offline functionality while reducing the data footprint of centralized servers.[3]

Example Use Case. Consider a user wearing a smartwatch that continuously collects data such as GPS position, step count, and heart rate in a SQLite database (the PDS on the right of Figure 1). While all this information may be relevant for the user, much of it is sensitive or too fine-grained for applications. For instance, heart rate might be smoothed to plot, and GPS positions may get mapped to OpenStreetMap (OSM) road segments, aggregating them to coarser-grained units in space and time, which reduces data volume. In SIDRA, such raw, detailed data remains local. If this fitness app maintains a leaderboard of the top 10 runners per city, only an aggregate (blue data in Figure 1), such as the daily step count or fastest track, needs to be shared centrally (after user consent, of course), in the MVs hosted on the server (right of Figure 1). Note that decentralized processing of Top-N queries can reduce communication and potential data leakage by only sending data if it would enter the current Top-N [38].

Beyond leaderboards, the fitness website (and fitness app) might display, for example, a world map showing the average number of runners per OSM road segment, along with their speed and heart rate distributions. In places with very few inhabitants or app users, such averages could disclose sensitive personal information (such as a heart condition), e. g. if only one person was running in that time/space. As an additional privacy measure, SIDRA therefore allows data architects to specify a *minimum aggregation level* of e. g. 10, such that only those aggregate results would be populated if these were gathered from 10 distinct users.

2.1 Architecture

SIDRA follows a three-tier architecture, shown in Figure 2 formed by PDSs, a privacy-enforcing staging area, and a centralized warehouse. Each PDS, composing the **decentralized PDSs** layer (left), is a small database owned by a single user, containing only that user's data (red), and each has the same schema. The aggregated, partial results (blue) computed through IVM form a continuous stream of data changes, which are gathered in a **staging area** (middle): a buffer-like area that consolidates deltas and emits them when they are considered privacy-friendly. The SIDRA staging area has two main roles: firstly, it temporarily stores incremental, partial results, to either propagate these to the centralized tier, keep them in the buffer for longer, or permanently discard them. Second, it protects intermediate data from central database users who might attempt to view the data before it reaches its final stage within the pipeline. Updates, for example, might arrive asynchronously and not yet meet the minimum granularity requirements. Neither PDS users nor database users should have access to the staging area, respectively because it contains data from other users, and because its data might not be privacy compliant yet.

From the staging area, sanitized deltas (green) finally migrate to the **centralized warehouse** (right) in server-side materialized views for warehouse users to perform data analysis. Data continuously flows between these components and incrementally updates previous results, effectively modeling decentralized analytical processing as a *streaming IVM* problem.

Coming back to the previous example, the fleet of SQLite databases represents the decentralized PDSs, while the average heart rate data is hosted on the centralized warehouse. Smartwatches can compute partial aggregates every hour, but these intermediate values might not be immediately needed for analytics. At the same time, computing the minimum aggregation check requires a global view of the union of all users' PDSs. The staging area — which can be, for example, a PostgreSQL database hosted by a separate foundation — addresses this problem: it collects partial updates and holds them until the data is "ready" to be emitted, for example at the end of the day[4].

S-I-D-R-A stands for Streaming, Incremental & Decentralized Responsible Architectures, and while we already motivated the latter, we will do so now for the first two aspects.

Streaming. In SIDRA, updates from decentralized sources are processed continuously as they arrive, maintaining an evolving analytical state. If a stream update, therefore, added the heart rhythm sequence of a single person to a central aggregate that keeps rhythm distributions, which may already hold contributions from 10 users, then superficially, the updated value aggregating now 11 users would not violate the minimum aggregation level of 10. However, an observer of the centralized MV could still reconstruct the heart data of that single person by observing what changed. This

[2]See e.g. https://www.localfirstconf.com

[3]We do not strictly assume a PDS being on a mobile device or laptop – multi-tenant cloud storage can also be used, or a local database with cloud replication or backups.

[4]In this paper, we picture the staging area as a relational system; however, the SIDRA design does not exclude NoSQL or streaming engines, as long as they support a SQL-compatible dialect.

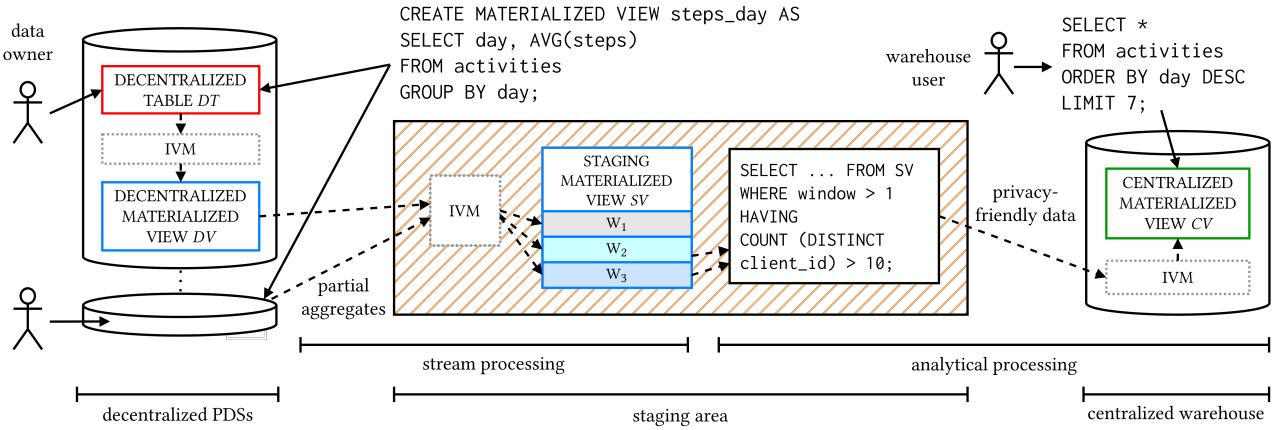

Figure 2: The SIDRA architecture. Raw data (red) never leaves the PDS: from decentralized tables DTs, a decentralized materialized view (DMV) is continuously updated via IVM, producing aggregated results (blue). These are then transmitted (dashed arrows) in a stream-like manner to a staging materialized view (SMV) within the staging area (middle), which partitions incoming updates into windows and maintains them. Here, additional privacy checks are applied to non-expired windows (W_2 and W_3); for instance, ensuring that each published window contains contributions from a sufficient number of clients. Finally, privacy-compliant deltas (green) are forwarded to the centralized warehouse (right), where the centralized materialized view (CMV) is updated through IVM and becomes available for the warehouse user.

is why *each individual update* must meet the privacy constraints, which in turn motivates why streaming is a first-class concept in SIDRA.

SIDRA uses a streaming model where tuples obtain a *generation timestamp* that maps them into a specific window of fixed length. Windows can be refreshed multiple times, possibly leading to refinements of aggregated values of the same window, as introduced in DataFlow [6].

The fact that the staging area may sometimes withhold stream updates to the centralized MVs, means that centralized materialized aggregates are typically incomplete. This can also result from the fact that certain users may not have been online, and thus, did not send updates, or because updates were discarded due to an expired timestamp. SIDRA provides observability into this, through the concepts of *completeness* of aggregates and the *responsiveness* in a window, explained in §4. The responsiveness for a window, for example, is the total fraction of users that contacted the staging area during that time.

IVM. In SIDRA, IVM captures the principle that analytical results are refreshed through small, continuous deltas rather than full re-computations. The centralized warehouse cannot contact or access the decentralized PDSs — since these are managed exclusively by end-users and could also be offline during longer periods. For instance, the PDS would only be active while an app is used; or even when used, the device it runs on could be offline network-wise. Hence, only when an end-user accesses her PDS, is there access to the raw data (via authentication/decryption with user-only credentials), and only at that moment can she compute and communicate changes to the staging area, in a push-based manner. By compiling a *declarative materialized view* into per-client partial and incremental delta streams, SIDRA ensures that raw event records (the original personal data) remain within each PDS. This motivates why IVM is a fundamental concept in SIDRA: end-users send deltas on their own initiative, and these deltas eventually make their way to central servers into MVs. IVM is therefore executed in every component of the SIDRA architecture, as seen in Figure 2.

2.2 Privacy Model

SIDRA's privacy model assumes an honest-but-curious attacker who may inspect the data in the centralized warehouse and infer information from it (or its changes), but following the protocol correctly. We do not require the staging area to be a fully trusted proxy. In practice, several deployment models are possible: (i) co-location with cloud providers, where the staging runs in the same public cloud as the centralized warehouse but with reduced trust assumptions; (ii) consortium-hosted staging, operated by a group of data controllers, regulators, or non-profits; or (iii) enterprise self-hosting within a private server. Moreover, staging logic can be run inside confidential computing enclaves or integrated with multiple servers running MPC protocols, further mitigating trust. In more adversarial settings, the same mitigation mechanisms also protect against an outright system breach. Thus, within our threat model, users need only trust that the staging area enforces the declared privacy annotations and security policies. We further assume the database architect correctly defines the SIDRA schema and its keywords, ensuring that all aggregated queries remain privacy-compliant; this can be independently verified through audits, cryptographic attestation, or publicly exposed interfaces. However, because the sensitive information never leaves the PDSs, even a fully compromised staging area cannot reveal the underlying raw inputs, effectively decoupling analytical value from ownership. This represents an improvement over traditional centralized systems, where a single fallacy can expose the entire dataset. A separate threat arises from an attacker compromising a user's PDS. In this case, only that user's data is affected: the breach does not propagate to other participants or to the aggregated results. Further, disk-based encryption can protect data at rest.

3 SIDRA SQL

[todo add text2sql snowflake reference + bigquery] In this section, we present SIDRA SQL, the declarative interface to define the data placement, privacy rules, and streaming computation intervals. SIDRA SQL addresses RQ1.1 by enabling developers to declaratively specify privacy-aware data architectures directly in SQL. From a single specification, SIDRA SQL automatically derives the components and their privacy attributes across the entire SIDRA architecture, providing expressive control over diverse privacy use cases.

3.1 Table and View Scope

To determine which component of our architecture should host a specific table or view, we introduce new scopes: *centralized* vs. *decentralized*, and expand these concepts with *replicated* elements. The SIDRA SQL syntax is highlighted in Listing 3.1.

Listing 1: SIDRA SQL grammar for table and view placement.

```
<create table statement> ::=
    CREATE <scope> TABLE <table name>
    ( <column definition> [, <column definition> ]* )
<scope> ::=
    [ CENTRALIZED | DECENTRALIZED ]
<create view statement> ::=
    CREATE [<view scope>] VIEW  <view name>
    AS <view query> ;
<view scope> ::=
    [ <scope> [MATERIALIZED] | REPLICATED MATERIALIZED ]
```

Decentralized tables *DTs* store individual user information within personal data stores. From a centralized point of view, a decentralized table could be viewed as a horizontally partitioned table with an implicit user ID as the partitioning key. However, this is a conceptual notion only, since decentralized tables (or views) cannot be queried centrally. Rather, each individual user manages her own table "partition" in her own PDS. Each PDS is isolated, (potentially) encrypted, and requires user authentication; thus, it can only be queried by the user who owns it. Decentralized tables, therefore, provide data minimization and ownership.

Centralized tables *CTs* reside on the central warehouse and store data that is globally accessible within that environment. Unlike *DTs*, *CTs* are managed and queried entirely at the server side and represent conventional relational tables, without any implicit partitioning or privacy isolation.

Decentralized views *DVs* are defined over decentralized tables (or views) only, and are stored in each PDS. For each user, they are based only on the user's data. Decentralized *materialized* views *DMVs* are maintained on changes (deltas) in decentralized tables from that PDS, and therefore still contain information for one single individual. *DVs* are the building blocks for creating *CMVs* (centralized materialized views, introduced later), and integrate streaming semantics, establishing how often the upstream query should be refreshed. Decentralized materialized views maintain (typically aggregated) query results in each PDS, partitioned by their *window* of belonging [todo-reread dataflow paper; think about parameter imposing how many windows to keep].

Centralized views *CVs* are views available in the centralized warehouse, and are defined over *CTs* or *CVs*. Centralized *materialized* views *CMVs* can, additionally, use data from *DMVs*. Here, further aggregation measurements can be specified, ensuring that *CMVs* only contain data that is considered sufficiently anonymized. [todo what happens if we have a SV but no CV on top?]

To monitor the health and timeliness of data ingestion, SIDRA automatically derives observability columns on top of *CMVs*. These metrics, highlighted in table 1, quantify how decentralized clients contribute to each window and help assess trade-offs between freshness and completeness. Specifically, *responsiveness* captures the fraction of clients that successfully emit deltas during a window, serving as an indicator of participation across the network. *Completeness* measures the fraction of tuples that are successfully published to *CMV*, excluding those discarded due to lateness or failure to meet the minimum aggregation requirements. Finally, *buffer size* represents the proportion of tuples that remain in staging views — data that has been received but not yet flushed to the central warehouse — providing a measure of backlog and synchronization delay.

Table 1: Observability Metrics added to *CMVs*.

Column	Description
`responsiveness(w)`	% of clients emitting results
`completeness(w)`	% of insertions in *CMV* (not discarded)
`buffer_size(w)`	% of rows still in the buffer

Replicated materialized views *RMVs* are centralized MVs, that may e. g. contain lookup tables or overviews of the analyzed data, including leaderboards or dashboards. They are maintained on the central server, but a read-only copy of them is kept inside each PDS.

While **staging materialized views *SMVs*** also exist in SIDRA, we deliberately exclude them from our grammar: they are not defined through SQL but rather generated by the SIDRA compiler, as discussed later.

3.2 Streaming Keywords

In SIDRA, streaming keywords define how *DMVs* communicate incremental updates to the central server. Each *DMV* produces a stream of deltas — tuples representing insertions, deletions, or updates. These carry not only aggregated values, but also system metadata: notably a window, encoding a generation-time interval, a client ID to track the originating source in an anonymized way, and an action that specifies whether the row inserts or removes data. The resulting tuples are stored in *staging materialized views* *SMVs*, relational tables that mirror the schema of their originating *DMVs* and extend it with the system columns listed in Table 2. *SMVs* buffer deltas per window and periodically flush them to the central warehouse with IVM.

Listing 2: SIDRA SQL streaming semantics. `MINIMUM AGGREGATION` is explained in the following subsection.

```
<decentralized view definition> ::=
    CREATE DECENTRALIZED VIEW <view_name>
```

Table 2: System columns added to *SMVs* by SIDRA.

Column	Description
`window`	Logical window derived from generation time
`client_id`	Identifier of the originating client
`action`	Operation type (`INSERT` or `DELETE`)

```
    AS <query> <DMV streaming keywords>;
<DMV streaming keywords> ::=
    WINDOW <integer literal>
    TTL <integer literal>
    REFRESH <integer literal>
<centralized view definition> ::=
    CREATE CENTRALIZED VIEW <view_name>
    AS <query> <CMV streaming keywords>
    [MINIMUM AGGREGATION <integer literal>];
<replicated view definition> ::=
    CREATE REPLICATED VIEW <view_name>
    AS <query> <CMV streaming keywords>
    [MINIMUM AGGREGATION <integer literal>];
<CMV streaming keywords> ::=
    REFRESH <integer literal>
```

Window represents how long each window of generation time should be, as well as an additional grouping key for the incremental query outcomes. A *CMV* will provide answers for each window, and may update the intermediate results multiple times, as defined by the `REFRESH` explained later. We arbitrarily choose seconds as a unit of measurement for our prototype, and tumbling windows to split the data, although this could be extended in the future. Streaming IVM updates are mapped to a window based on their generation time, and either published to *CMV* or discarded.

Refresh empowers data architects with more freedom in the materialization intervals, which do not necessarily align with windows. On the decentralized component, `REFRESH` represents the IVM processing interval, as well as how often PDSs are to communicate intermediate results with the server. In other words, every time a `REFRESH` interval elapses, a *DMV* is updated, and the resulting deltas are streamed to the staging area.

In the staging area and the central warehouse, `REFRESH` is a trigger to materialize the query output. Refreshing is useful to provide (more) up-to-date metrics, which refine over time during a window as inputs from the *DMVs* are collected — for example, if the *DMV* in Listing 4 were grouped by minute rather than hour, frequent refreshes would enable more fine-grained analytics. Each staging view inherits its `WINDOW` and `REFRESH` parameters from its corresponding *DMV* to ensure consistent synchronization. In contrast, *CMVs* may independently define their own refresh intervals.

TTL represents the time-to-live of a change tuple, specified in time units (minutes? todo discuss). With the asynchronous nature of our system, late arrivals are bound to happen, lagging one or more window units. This creates the need for a cutoff defining whether a late arrival can still be considered useful for the final result or should be discarded.

3.3 Privacy Keywords

We introduce privacy keywords to specify what can be learned by other parties from query results, allowing declarative management of information leak across PDSs, staging area, and *CMVs*.

Listing 3: SIDRA SQL privacy semantics.

```
<query> ::=
    SELECT <projection_item> { , <projection_item> }*
    <query_rest>
<projection_item> ::= <expr> [ <annotation> ]
<annotation> ::= '1' | '2' | '3(' <dp_params> ')' | '4'
<dp_params> ::= 'eps=' <number>
    [ ',' 'sens=' <number> ]
    [ ',' 'lower=' <number> ',' 'upper=' <number> ]
```

We introduce *privacy levels*, described in Table 3: annotations in *DMV* definitions represent the level of privacy between *DMVs* and the staging area, while in *CMVs* and *RMVs* they represent privacy between the staging area and *CMVs*. Each annotation is a numeric privacy level (1-3), which defines the degree of aggregation or privacy applied to the column, as well as how the attribute should be processed and what information can be shared. If a column is annotated with privacy level N, it automatically inherits all lower levels 1 through $N-1$ for that direction. For example, a column annotated with S3 is considered to also satisfy S1 and S2 constraints [todo double check].

Table 3: Column privacy levels and the knowledge exposed.

Privacy level	Knowledge gained from column
`NULL`	Unprocessed plaintext
1	Individual-level aggregated plaintext
2	Multi-individual-level aggregated plaintext
3	Differentially private plaintext
4	Restricted plaintext

Level 1 (for PDS only) must be aggregated locally before leaving the source, ensuring that raw individual records are never exposed.

Level 2 requires minimum aggregation: each aggregate value must include contributions from multiple individuals, preventing the reconstruction of individual records from grouped data. The grouping for the minimum aggregation count is automatically applied to all columns with this annotation.

Level 3 is protected using DP. Detailed DP mechanisms are described in §5.2.

NULL (default setting) carries no privacy guarantees: columns are exposed in raw plaintext and can be freely used in any queries.

Minimum aggregation is a further restriction to staging views, imposing an additional minimal number of elements in each group of an aggregation. It represents the minimum number of clients for each distinct subset of keys in each window, establishing an additional privacy measure to hinder the reconstruction of individuals based on aggregate data. For columns with privacy level 2, minimum aggregation is automatically enforced.

Listing 4: Defining a decentralized MV with aggregated step counts per day, a centralized MV with steps per (city, day), and a replicated MV with the 10 most active users. We assume that the nickname is a non-identifying field. todo double check numbers

```
CREATE DECENTRALIZED MATERIALIZED VIEW daily_steps_user
AS (SELECT nickname,
        osm_pos_to_city(latitude, longitude) AS city 2,
        DATE(start_time) AS day 2,
        SUM(steps) AS total_steps 1
    FROM activity_sessions
    WHERE activity_name = 'run' OR activity_name = 'walk'
    GROUP BY nickname, city, day)
REFRESH 14400 --- every 4h (client and server-side)
WINDOW 86400 --- emit one result per day
TTL 172800; --- ignore updates that are > 48 hours late

CREATE CENTRALIZED MATERIALIZED VIEW daily_steps
AS (SELECT city 2, day 2, SUM(total_steps) AS total_steps
    FROM daily_steps_user
    GROUP BY city, day)
REFRESH 43200 --- refresh every 12h (server-side)
MINIMUM AGGREGATION 10; --- minimum number of users per
     city, day

CREATE REPLICATED MATERIALIZED VIEW leaderboard
AS (SELECT nickname 1, SUM(total_steps) AS daily_steps
    FROM daily_steps_user
    WHERE nickname IS NOT NULL --- consenting users
    GROUP BY nickname
    ORDER BY total_steps DESC LIMIT 10)
REFRESH 86400; --- refresh daily (update leaderboard)
```

4 SIDRA SEMANTICS

4.1 [Dandan:] SIDRA Schema and Relation

In SIDRA, each PDS has unrestricted access to its own data. To precisely control what information can be disclosed to the staging area and the central server, we introduced a *column-level privacy model* in §2 that provides a structured mechanism for specifying and enforcing privacy rules at the attribute level. To formally define this model, we first introduce the concept of the SIDRA schema and the SIDRA Relation. Throughout the following definitions, we use D (i.e. Decentralized) to denote the PDS, S to denote the staging area, and C to denote the central server.

We define the SIDRA schema as the union of the schemas associated with the PDS, the staging area, and the central server.

Definition 1 (**SIDRA Schema**). *A SIDRA Schema (denoted as SRS) is defined as:*

$$SRS = SRS^D \cup SRS^S \cup SRS^C,$$

where:

- $SRS^D = \{RS^D_1, \ldots, RS^D_{num_D}\}$, *where each* RS^D_i *denotes a relation schema for* $1 \leq i \leq num_D$*;*
- $SRS^S = \{RS^S_1, \ldots, RS^S_{num_S}\}$, *where each* RS^S_i *denotes a relation schema for* $1 \leq i \leq num_S$*;*
- $SRS^C = \{RS^C_1, \ldots, RS^C_{num_C}\}$, *where each* RS^C_i *denotes a relation schema for* $1 \leq i \leq num_C$*.*

We further require that SRS^D *contains an attribute id, which identifies the PDS from which each row originates. Similarly,* SRS^S *contains an attribute* id' *that serves the same purpose. It is not necessary that* $id' = id$*; for instance,* id' *may be a hashed transformation of id.*

Next, we formally define the notion of a *SIDRA Relation*. This definition adopts the time-based relation models introduced by Arasu et al. in CQL [8] : we let $\mathcal{T} = \{0, 1, 2, \ldots\}$ denote a discrete, ordered time domain, where each element of $\mathcal{T}$ represents a single time instant. A stream S is a possibly infinite multiset (bag) of elements $\langle s, \tau \rangle$, where s is a tuple conforming to its schema and $\tau \in T$ is its timestamp. A relation R is a mapping from each time instant $\tau \in T$ to a finite (but unbounded) multiset of tuples, and $R(\tau)$ represents the current contents of the relation at τ. In SIDRA, both base tables and derived views are modeled as such time-varying relations [5], whose state evolves as new tuples are produced or deleted over time. We extend this concept by incorporating two additional notions: (i) a *relation keeper*, which identifies the entity responsible for the relation, and (ii) a relation index corresponding to that keeper. These extensions are essential, as SIDRA enforces privacy constraints based on the entity responsible for data transmission or storage.

Definition 2 (**SIDRA Relation**). *A SIDRA Relation, denoted by R, is a mapping from a triple consisting of a relation keeper* $x \in \{D, S, C\}$*, an index* $i \in [1, num^x]$*, and a time instant* $\tau \in \mathcal{T}$ *that is,* (x, i, τ) *to a bag of tuples.*

Each tuple r in this bag conforms to the corresponding relation schema such that $\mathrm{Attr}(r) = \mathrm{Attr}(SRS^x_i)$*.*

For a given triple (x, i, τ), we denote by $R[x, i, \tau]$ the bag of tuples associated with that triple. For two time instants $a, b \in \mathcal{T}$ such that $a \leq b$, we define $R[x, i, a \leq \tau \leq b]$ as the union $\bigcup_{\tau=a}^{b} R[x, i, \tau]$.

4.2 [Dandan:] SIDRA Privacy Semantics

In this subsection, we present a formal column-level privacy model that applies to arbitrary schemas. The model consists of two components: a *column-level privacy configuration* and a *privacy-aware relational algebra*. The first component provides a formal framework for specifying policies that govern what information may be disclosed about each column in the schema. The second component ensures that any query evaluated over a privacy-configured schema adheres to the privacy guarantees enforced by its corresponding column-level configuration. In SIDRA, we instantiate this privacy configuration model separately for SRS^D and SRS^S. Tuples in the schema SRS^S are produced by executing privacy-aware queries over tuples in SRS^D, and tuples in the schema SRS^C are produced by executing privacy-aware queries over tuples in SRS^S. In this way, our model enables fine-grained column-level control over what information about PDS data may be revealed to the staging area, and what information collected in the staging area may in turn be revealed to the central server.

4.2.1 ***Notations.*** We use sch to denote an arbitrary schema, which may contain a *privacy-unit attribute* punit-the attribute that identifies the entity whose data the aggregation and differential privacy protections are intended to cover. In the context of SIDRA, punit corresponds to the identifier of a PDS, that is, either the attribute *id* in SRS^D or the attribute id' in SRS^S. We write $\mathrm{Attr}(\mathrm{sch})$ for the set of all attributes in sch. We use attrs to denote a list of

$\text{Check}_1(\text{attrs}, \alpha', B)$:
if attrs $\subseteq \big((\text{Attr}(B) \setminus \text{Attr}(\text{sch})) \cup \text{Attr}_0(\text{sch}) \cup \text{RAttr}_1(\text{sch})\big)$ and $\alpha' \notin (\text{Attr}(\text{sch}) \cup \text{RAttr}_1(\text{sch}))$ **then**
  return true
end if
return false

$\text{Check}_2(f, \text{attrs}, \alpha', B)$:
if $f = \text{NULL}$ and $|\text{attrs}| = 1$ and attrs $\subseteq (\text{Attr}_2(\text{sch}) \cup \text{Attr}_3(\text{sch}))$ and $\alpha' = \text{NULL}$ **then**
  return true
end if
return false

$\text{Check}_3(\text{gattrs}, \text{Agg}, \text{GA}, \alpha, \alpha')$:
if $(\text{gattrs}, \text{Agg}) \in \text{GA}[\alpha]$ and $\alpha' \notin (\text{Attr}(\text{sch}) \cup \text{RAttr}_1(\text{sch}))$ **then**
  return true
end if
return false

$\text{SanitizeLevel1}(B)$:
for each $\alpha \in (\text{Attr}(B) \cap \text{Attr}_1(\text{sch}))$ **do**
  $(-, \text{policy}) \leftarrow PC_{\text{sch}}[\alpha]$, $\{\text{Rename}, \text{Minimize}(\cdot)\} \leftarrow \text{policy}$
  $\widehat{\alpha} \leftarrow \text{Rename}[\alpha]$, $B \leftarrow \text{Minimize}(\alpha, \widehat{\alpha}, B)$
end for
return B

$\text{SanitizeLevel23}(\text{gattrs}, \text{Agg}, \alpha, B)$:
$(-, \text{policy}) \leftarrow PC_{\text{sch}}[\alpha]$
Extract the components punit and $\text{MinimizePreAgg}(\cdot)$ from policy.
$B \leftarrow \text{MinimizePreAgg}(\text{gattrs}, \text{Agg}, \alpha, \text{punit}, B)$
return B

$\text{SanitizeLevel4}(B)$:
for each $\alpha \in (\text{Attr}(B) \cap \text{Attr}_4(\text{sch}))$ **do**
  $(-, \text{policy}) \leftarrow PC_{\text{sch}}[\alpha]$, $\text{Removal}(\cdot) \leftarrow \text{policy}$, $B \leftarrow \text{Removal}(\alpha, B)$
end for
return B

(Privacy-aware Selection) $\sigma^{p}_{bool_f(\text{attrs})}(B) \rightarrow B'$:
Validity Check:
if $\text{Check}_1(\text{attrs}, \text{NULL}, B) = \text{false}$ **then**
  return NULL
end if
Sanitizing Procedure (level 1 and level 4):
$B \leftarrow \text{SanitizeLevel1}(B)$, $B \leftarrow \text{SanitizeLevel4}(B)$
Execution:
return $B' \leftarrow \sigma_{bool_f(\text{attrs})}(B)$

(Privacy-aware Projection) $\pi^{p}_{f_1(\text{attrs}_1)\rightarrow\alpha'_1, \ldots, f_\iota(\text{attrs}_\iota)\rightarrow\alpha'_\iota}(B) \rightarrow B'$:
Validity Check:
for $i = 1$ to ι **do**
  if $\text{Check}_1(\text{attrs}_i, \alpha'_i, B) = \text{false}$ and $\text{Check}_2(f, \text{attrs}_i, \alpha'_i, B) = \text{false}$ **then**
    return NULL
  end if
end for
Sanitizing Procedure (level 1):
$B \leftarrow \text{SanitizeLevel1}(B)$
Execution:
return $B' \leftarrow \pi_{f_1(\text{attrs}_1)\rightarrow\alpha'_1, \ldots, f_\iota(\text{attrs}_\iota)\rightarrow\alpha'_\iota}(B)$

(Privacy-aware Aggregation) $\gamma^{p}_{\text{gattrs}, \text{Agg}_1(\alpha_1)\rightarrow\alpha'_1, \cdots \text{Agg}_\iota(\alpha_\iota)\rightarrow\alpha'_\iota}(B) \rightarrow B'$:
for $i = 1$ to ι **do**
Validity Check:
  $(\text{level}, \text{policy}) \leftarrow PC_{\text{sch}}[\alpha_i]$
  if $\text{level} = 2$ **then**
    $\{\text{punit}, \text{GA}, \text{MinimizePreAgg}(\cdot)\} \leftarrow \text{policy}$.
  else if $\text{level} = 3$ **then**
    $\{\text{punit}, \text{GA}, \text{MinimizePreAgg}(\cdot), \text{DPpara}, \text{DPAgg}(\cdot)\} \leftarrow \text{policy}$
  elsereturn NULL
  end if
  if $\text{Check}_3(\text{gattrs}, \text{Agg}_i, \text{GA}, \alpha_i, \alpha'_i) = \text{false}$ **then**
    return NULL
  end if
end for
Sanitizing Procedure (level 1):
$B \leftarrow \text{SanitizeLevel1}(B)$
for $i = 1$ to ι **do**
Sanitizing Procedure (level 2 and level 3):
  $B_i \leftarrow \text{SanitizeLevel23}(\text{gattrs}, \text{Agg}, \alpha, B)$
Execution:
  if $\text{level} = 2$ **then**
    $B'_i \leftarrow \gamma_{\text{gattrs}, \text{Agg}_i(\alpha_i)\rightarrow\alpha'_i}(B_i)$
  else if $\text{level} = 3$ **then**
    $B'_i \leftarrow \text{DPAgg}(\text{gattrs}, \text{Agg}_i, \alpha_i, \text{punit}, \text{DPpara}, B_i)$
  end if
  if $i = 1$ **then**
    $B' \leftarrow B'_i$
  else
    $B' \leftarrow B' \bowtie B'_i$
  end if
end for
return B'

(Privacy-aware Join) $\bowtie^{p}_{bool_f(\text{attrs}_1, \text{attrs}_2)}(B_1, B_2) \rightarrow B'$:
Validity Check:
if $\text{Check}_1(\text{attrs}_1, \text{NULL}, B_1) = \text{false}$ or $\text{Check}_1(\text{attrs}_2, \text{NULL}, B_2) = \text{false}$ **then**
  return NULL
end if
Sanitizing Procedure (level 1 and level 4):
$B_1 \leftarrow \text{SanitizeLevel1}(B_1)$, $B_1 \leftarrow \text{SanitizeLevel4}(B_1)$
$B_2 \leftarrow \text{SanitizeLevel1}(B_2)$, $B_2 \leftarrow \text{SanitizeLevel4}(B_2)$
Execution:
return $B' \leftarrow \bowtie_{bool_f(\text{attrs}_1, \text{attrs}_2)}(B_1, B_2)$

(Privacy-aware Final Sanitization) $F^{p}(B) \rightarrow B'$:
$B \leftarrow \text{SanitizeLevel1}(B)$, $B \leftarrow \text{SanitizeLevel4}(B)$
for each $\alpha \in \text{Attr}(B) \cap (\text{Attr}_2(\text{sch}) \cup \text{Attr}_3(\text{sch}))$ **do**
  $B \leftarrow \text{Removal}(\alpha, B)$
end for
return $B' \leftarrow \delta(B)$

Figure 3: Dandan:Privacy-aware Relational Algebra over Schema sch

attributes, gattrs to denote specifically a list of grouping attributes, and Agg to denote an aggregation operator.

The privacy configuration for sch associates each attribute $\alpha \in \text{Attr}(\text{sch})$ with a numeric privacy level in $\{0, 1, 2, 3, 4\}$ and a corresponding privacy-enforcing policy. We write $\text{Attr}_i(\text{sch})$ to denote the set of attributes in sch whose privacy level is i.

We write B for a bag of tuples whose schema does not necessarily coincide with sch, and we write $\text{Attr}(B)$ for the set of attributes appearing in the schema underlying B.

4.2.2 ***Column-level Privacy Configuration***. As described above, our privacy model associates each column with a privacy level together with its corresponding privacy-enforcing policy. Each such policy comprises a collection of dedicated structures and functions. The structures provide declarative specifications that coarsely determine which operations are permitted on sensitive attributes and which parameters may be used. The functions, in turn, operationalize these constraints and may perform more fine-grained data-minimization transformations.

We introduce the privacy-enforcing structures Rename, GA, and DPpara, as well as the auxiliary notation $\text{RAttr}_1(\text{sch})$.

Aliases for level-1 attributes. For each attribute $\alpha \in \text{Attr}_1(\text{sch})$, a unique alias $\widehat{\alpha}$ is required. We write Rename for the mapping that assigns to each $\alpha \in \text{Attr}_1(\text{sch})$ its alias $\widehat{\alpha}$. As we will show, aliasing prevents direct use of α. Also, we write $\text{RAttr}_1(\text{sch})$ for the set of all such aliases.

Permitted grouping–aggregation combinations. For each attribute $\alpha \in \text{Attr}_i(\text{sch})$ with $i \in \{2, 3\}$, one must restrict valid grouping-aggregation combinations. Formally, we write GA for the mapping that assigns to each $\alpha \in \text{Attr}_2(\text{sch}) \cup \text{Attr}_3(\text{sch})$ a set of pairs of the form $\{(\text{gattrs}, \text{Agg}), \ldots\}$, where each pair $(\text{gattrs}, \text{Agg})$

indicates that the aggregation $\mathsf{Agg}(\alpha)$ is permitted over the groups defined by gattrs. In particular, we require that each set of grouping attributes satisfies $\mathsf{gattrs} \subseteq \mathsf{Attr}_0(\mathsf{sch}) \cup \mathsf{RAttr}_1(\mathsf{sch})$; that is, only attributes with privacy level 0 and aliases of attributes with privacy level 1 may be used as grouping attributes.

Differential privacy parameters. Given an attribute $\alpha \in \mathsf{Attr}_3(\mathsf{sch})$, a pair $(\mathsf{gattrs}, \mathsf{Agg}) \in \mathsf{GA}[\alpha]$, and the privacy unit punit, we use a mapping DPpara to associate the aggregation $\mathsf{Agg}(\alpha)$ over the groups defined by gattrs with a tuple of differential privacy (DP) parameters $(\epsilon_{dp}, \delta_{dp}, M_r, M_g, v_{\min}, v_{\max})$. Here, ϵ_{dp} and δ_{dp} denote, respectively, the privacy budget allocated to the aggregation and its associated DP failure probability. The parameter M_r specifies the maximum number of rows that a single punit may contribute to $\mathsf{Agg}(\alpha)$ within any one group, while M_g specifies the maximum number of distinct groups to which a punit may contribute. The interval $[v_{\min}, v_{\max}]$ denotes the clipping bounds applied to the aggregation attribute. Formally,

$$\mathsf{DPpara}[\mathsf{gattrs}, \mathsf{Agg}, \alpha, \mathsf{punit}] = (\epsilon_{dp}, \delta_{dp}, M_r, M_g, v_{\min}, v_{\max}).$$

These parameters bound the sensitivity of the aggregation and determine the noise magnitude used in the DP mechanism.

We introduce the privacy-enforcing functions $\mathsf{Minimize}(\cdot)$, $\mathsf{MinimizePreAgg}(\cdot)$, and $\mathsf{DPAgg}(\cdot)$. Each function operates on a bag of tuples and produces a new bag or set as its output.

Attribute Minimization: $\mathsf{Minimize}(\alpha, \widehat{\alpha}, B) \rightarrow B'$. This function takes as input an attribute α, an alias $\widehat{\alpha}$, and a bag of tuples B containing α. It produces a minimized bag B' based on the values of α, in which the attribute name α is replaced by its alias $\widehat{\alpha}$. For example, each value of α in B may be replaced with a minimized (e.g., generalized, approximated, or hashed) version of the original value, and some rows containing specific values — such as outliers — may be removed. In standard relational algebra, this can be mapped to a selection followed by a projection with an expression:

$$B' = \Pi_{\alpha' = f_\alpha(\alpha), \beta_1, \ldots, \beta_k}\left(\sigma_{p_\alpha(\alpha)}(B)\right),$$

where:

- $\beta_1, \ldots, \beta_k$ are the other attributes in B,
- f_α is the minimization function applied to attribute α
- $p_\alpha(\alpha)$ is a predicate filtering out tuples that should be removed (e.g., outliers or sensitive values).

Attribute-pre-aggregation Minimization: $\mathsf{MinimizePreAgg}(\mathsf{gattrs}, \mathsf{Agg}, \alpha, \mathsf{punit}, B) \rightarrow B'$. This function takes as input grouping attributes gattrs, an aggregation operator Agg, aggregation attribute α, and a privacy-unit attribute punit, and a bag of tuples B containing attributes $\mathsf{gattrs} \cup \{\alpha, \mathsf{punit}\}$. It produces a minimized bag of tuples B' by removing selected rows from B, in order to control the set of values over which each group is permitted to apply $\mathsf{Agg}(\alpha)$. The selection may, but is not required to, depend on the contributions of each punit. For example, the policy may require that a group identified by gattrs may perform a $\mathsf{sum}(\alpha)$ only if it contains contributions from more than ten distinct privacy units. In this case, all rows belonging to groups with fewer than ten contributing punit values are removed from B when producing B'. Note that the field punit may be set to NULL, indicating that no per-unit contribution limits apply.

Attribute Aggregation with Differential Privacy: $\mathsf{DPAgg}(\mathsf{gattrs}, \mathsf{Agg}, \alpha, \mathsf{punit}, \mathsf{DPpara}, B) \rightarrow B'$. The function takes grouping attributes gattrs, an aggregation operator Agg, an aggregation attribute α, a privacy-unit attribute punit, the differential privacy parameters DPpara, and a bag of tuples B whose schema contains the attributes $\mathsf{gattrs} \cup \{\alpha, \mathsf{punit}\}$. The output B' consists of one tuple for each distinct group obtained by grouping B on gattrs, paired with the corresponding differentially private estimate of $\mathsf{Agg}(\alpha)$.

We note that existing differential privacy libraries can be used as building blocks for implementing DPAgg. Let c denote the number of distinct groups defined by gattrs in B. Given the parameters $(\epsilon_{dp}, \delta_{dp}, M_r, M_g, v_{\min}, v_{\max})$ specified in $\mathsf{DPpara}[\mathsf{gattrs}, \mathsf{Agg}, \alpha, \mathsf{punit}]$, with M_g updated to the smaller of its original value and c, together with the aggregation operator Agg and the array of values of α within each group that satisfy the contribution limits, the Google DP libraries [] can automatically compute the sensitivity and produce a noisy estimate of $\mathsf{Agg}(\alpha)$ for each group, with total privacy loss bounded by ϵ_{dp} and failure probability δ_{dp}. To implement DPAgg using the Google DP libraries, we first perform the appropriate pre-aggregation or filtering within each group to enforce the contribution limit M_r, and subsequently filter B across groups to enforce the limit M_g. We then invoke the Google DP library, as described above, to compute the differentially private aggregation for each group. Finally, each distinct value of gattrs appearing in B, together with its corresponding DP aggregation result, forms a tuple in B'.

Attribute Removal: $\mathsf{Removal}(\alpha, B) \rightarrow B'$. This function takes as input an attribute α and a bag B containing α, and outputs a new bag B' obtained by removing the entire column α.

Definition 3 (**Column-level Privacy Configuration for a schema**). *We define the Privacy Configuration for a schema* sch*, denoted as* PC_{sch}*, as a mapping: for each attribute* $\alpha \in \mathsf{Attr}(\mathsf{sch})$ *outputs*

$$PC_{\mathsf{sch}}[\alpha] = (\mathsf{level}, \mathsf{policy})$$

where $\mathsf{level} \in \{0, 1, 2, 3, 4\}$ *represents the privacy level,* policy *denotes the corresponding privacy-enforcing policy.*

The configuration satisfies the following rules.

- *If* $\mathsf{level} = 0$, *then* $\mathsf{policy} = \mathsf{NULL}$.
- *If* $\mathsf{level} = 1$, *then* $\mathsf{policy} = \{\mathsf{Rename}, \mathsf{Minimize}(\cdot)\}$.
- *If* $\mathsf{level} = 2$, *then* $\mathsf{policy} = \{\mathsf{punit}, \mathsf{GA}, \mathsf{MinimizePreAgg}(\cdot)\}$.
- *If* $\mathsf{level} = 3$, *then* $\mathsf{policy} = \{\mathsf{punit}, \mathsf{GA}, \mathsf{MinimizePreAgg}(\cdot), \mathsf{DPpara}, \mathsf{DPAgg}(\cdot)\}$.
- *If* $\mathsf{level} = 4$, *then* $\mathsf{policy} = \mathsf{Remove}(\cdot)$.

4.2.3 ***Privacy-Aware Relational Algebra***. We represent each query that can be executed over a privacy-configured schema sch as a relational algebra tree. However, standard relational algebra alone is insufficient to enforce the intended privacy constraints.

For example, classical relational algebra permits selection predicates over arbitrary attributes. Suppose that during the privacy configuration process, a column such as age is assigned privacy level 1, and the corresponding function $\mathsf{Minimize}(\cdot)$ transforms precise values into generalized representations, e.g., replacing an exact age of 22 with an interval $[20, 30]$. If standard relational algebra were applied directly to sch, it would still allow selection

predicates that compare the age attribute to specific values. Executing such a query would reveal that all tuples in the result share the same underlying age value, thereby violating the intended privacy guarantees. More generally, certain attributes may be restricted to support only a subset of aggregation queries, possibly conditioned on specific grouping attributes, and may additionally be subject to DP requirements. These constraints cannot be expressed or enforced using standard relational algebra operators.

To address this problem, we introduce an extension of standard relational algebra equipped with privacy-aware semantics that guarantees compliance with the column-level privacy requirements specified during configuration for all queries. Specifically, we define privacy-aware counterparts of the standard relational algebra operators–selection, projection, grouping, and join–denoted by σ^p, π^p, γ^p, and $\bowtie^p$, respectively. In addition, we introduce a dedicated final sanitization operator, denoted by $\digamma^p$, which enforces any remaining privacy constraints prior to result release. We retain the standard bag-based union and intersection operators, denoted by $\cup$ and $\cap$, respectively. Thus, the operator set of our privacy-aware relational algebra is given by

$$\{\sigma^p, \pi^p, \gamma^p, \bowtie^p, \digamma^p, \cup, \cap\}.$$

The implementation of the dedicated privacy-aware operators may internally invoke the corresponding bag-based relational algebra operators σ, π, γ, and $\bowtie$, as well as the duplicate elimination operator δ, in conjunction with the privacy-enforcing functions defined in Section 4.2.2. Figure 3 presents the pseudocode for the proposed privacy-aware operators. In the pseudocode, we use $bool_f(\text{attrs})$ to denote an arbitrary Boolean function (i.e., a predicate) defined over an attribute list attrs, and $f(\text{attrs})$ to denote an arbitrary function over attrs. Each operator, except final sanitization operator $\digamma^p$, require three steps: validity check, sanitization, and execution.

Validity Checks. The operators σ^p, π^p, γ^p, and $\bowtie^p$ are each parameterized by accompanying expressions. To prevent privacy breaches, we apply operator-specific validity checks to these expressions. Specifically, predicates $bool_f$ associated with σ^p and $\bowtie^p$ are restricted to involve only level 0 attributes, aliases corresponding to level 1 attributes, and derived attributes introduced during query execution over the schema sch. The projection operator π^p allows arbitrary expressions, including renaming, over level 0 attributes, aliases for level 1 attributes, and derived attributes. In addition, for efficiency considerations, π^p may temporarily project level 2 and level 3 attributes; however, renaming over such attributes is disallowed, and they are guaranteed not to appear in the final query output. For aggregation, the operator γ^p verifies whether each grouping–aggregation combination is permitted by checking its membership in the structure GA. Finally, we require that any attribute introduced through renaming (in π^p or γ^p) must not coincide with any attribute in $\text{Attr}(\text{sch})$ or $\text{RAttr}_1(\text{sch})$, as these predefined attribute names are reserved for enforcing privacy controls. If the validity check for an operator does not pass, the operator returns NULL. The auxiliary validity check functions corresponding to these operators are denoted by $\text{Check}_1(\cdot)$, $\text{Check}_2(\cdot)$, and $\text{Check}_3(\cdot)$, as summarized in Figure 3.

Sanitization. After an operator passes the validity check, a sanitization procedure is applied to the input bag. The operators σ^p and $\bowtie^p$ perform minimization over level 1 attributes, as specified by the auxiliary function $\text{SanitizeLevel1}(\cdot)$, and remove level 4 attributes using $\text{SanitizeLevel4}(\cdot)$. The projection operator π^p only applies $\text{SanitizeLevel1}(\cdot)$, since the validity-check phase already ensures that level 4 attributes are not projected. The aggregation operator γ^p applies both $\text{SanitizeLevel1}(\cdot)$ and $\text{SanitizeLevel23}(\cdot)$, where the latter enables pre-aggregation minimization for level 2 and level 3 attributes. Note that $\text{SanitizeLevel1}(\cdot)$ not only performs attribute minimization, but also ensures that functions involving aliases for level 1 attributes can be evaluated correctly.

Execution. Following the validity-check and sanitization phases, the execution of σ^p, π^p, γ^p, and $\bowtie^p$ proceeds by directly invoking the corresponding standard bag-based relational algebra operators σ, π, γ, and $\bowtie$, respectively, using the same accompanying expressions. An exception arises when γ^p involves aggregation over level 3 attributes. In this case, the aggregation is performed by invoking the privacy-enforcing function $\text{DPAgg}(\cdot)$ with the specified differential privacy parameters.

Final Sanitization Operator $\digamma^p$. Operator-specific validity checks and sanitization steps alone are insufficient to guarantee that the final query result satisfies the configured privacy requirements when the top-level operator is not γ^p. To address this limitation, we introduce a final sanitization operator, denoted by $\digamma^p$, which is applied on top of the original query plan whenever the original top operator is not γ^p. The operator $\digamma^p$ applies $\text{SanitizeLevel1}(B)$ and $\text{SanitizeLevel4}(B)$ to the input bag B, removes all level 2 and level 3 attributes, and subsequently invokes duplicate elimination via $\delta(B)$. The purpose of duplicate elimination is to prevent the leakage of multiplicity (count) information associated with identical tuples. For example, when disclosing the exact number of individuals within a specific age range (e.g., 20–30) is disallowed, retaining duplicate tuples would allow an adversary to infer this information through a simple projection over the age interval, thereby violating the intended privacy configuration.

Privacy Analysis. Let q denote a query represented as a privacy-aware relational algebra tree whose top operator is either γ^p or $\digamma^p$. The semantics of our privacy-aware relational algebra guarantee that q never reveals raw values of level 1 attributes, never reveals unpermitted unaggregated or aggregated values of level 2 and level 3 attributes, and never reveals any values of level 4 attributes. Consequently, any such query is guaranteed to satisfy the configured column-level privacy requirements.

We further recommend that schema designers aim to minimize similarity between attributes in a privacy-configured schema, or alternatively, assign the same highest applicable privacy level to highly similar attributes. This practice mitigates the risk that sensitive values associated with an attribute assigned a higher privacy level could be indirectly inferred through another, highly similar attribute assigned a lower privacy level.

4.3 SIDRA Tables

We formally represent the tables and views residing in each PDS, the staging area, and the central server at time $a \in \mathcal{T}$.

Decentralized Tables. Suppose there are N clients, identified by $id_1, \ldots, id_N$. At time a, for each $1 \leq i \leq N$, the collection DTs_{id_i} of decentralized tables maintained by client id_i conforms to the

privacy-configured schema SRS^D and can be represented as

$$DTs_{id_i} = \{\sigma_{id=id_i}(R[D, j, a_0 \leq \tau \leq a])\}_{j=1}^{num_D},$$

where σ denotes the standard (non-privacy-aware) selection operator and a_0 is a value satisfying $0 \leq a_0 \leq a$.

We allow a_0 to be nonzero in order to support the deletion of outdated data by the clients.

Decentralized (Materialized) Views. Each client also maintains Decentralized Views (DVs) and Decentralized (Materialized) Views (DMVs), both of which conform to the privacy-configured schema SRS^S and are obtained by executing privacy-aware queries over the corresponding decentralized tables. That is, for client id_i,

4.4 SIDRA Tables

Decentralized tables. Each DT on client C_i is a pair

$$DT_i := (R_i, \text{schema}) \tag{1}$$

where R_i is the local instance of the table, i. e., the multiset of tuples stored on C_i, and schema $= \{c_1, c_2, \ldots, c_k\}$ defines the common table definition with its columns and their types.

Decentralized (materialized) views. DVs and $DMVs$ are time-varying relations defined by a query Q evaluated over data owned by a single PDS on C_i. Formally, Q denotes a relational algebra tree composed of operators from the set $\{\pi, \sigma, \bowtie, \gamma, \rho, \cup, \cap\}$. In SIDRA, Q defining a DMV must contain a projection with at least one aggregation operator γ, and may also include filters and joins between DTs, DVs, other $DMVs$, or $RMVs$ located in the same PDS C_i. Joins and unions must be children of aggregations; references to CTs or CVs are not permitted. Let $R_1^{(i)}, \ldots, R_n^{(i)}$ denote the downstream relations local to client C_i, where each $R_j^{(i)} \in \{DT, DV, DMV, RMV\}$. Each query Q is evaluated pointwise in time as

$$DMV_Q^{(i)}(\tau) = Q(R_1^{(i)}(\tau), \ldots, R_n^{(i)}(\tau)) \tag{2}$$

The resulting $DMV_Q^{(i)}$ is therefore a time-varying relation whose schema is augmented with the system-defined metadata attributes `client_id`, `window`, and `action`.

Each tuple $x \in DMV^{(i)}$ is assigned to a logical window w according to its generation timestamp $g_t(x)$, which records the time at which the corresponding DMV refresh completed. We define a deterministic windowing function in Eq. 3, where W_{size} is the declared window size `WINDOW`:

$$\omega : T \to \mathbb{N}, \quad \omega(g_t(x)) = \left\lfloor \frac{g_t(x)}{W_{size}} \right\rfloor \tag{3}$$

Furthermore, each tuple carries a `client_id`, a unique random identifier for its originating PDS, used to track client participation and to verify that groups meet the required aggregation threshold.

Staging materialized views *SMVs*. A SMV is a system-managed relation that collects the deltas produced by local $DMVs$ and prepares them for centralized ingestion. Operationally, the SMV behaves as an append-only, stream-like time-varying relation: when a new delta is emitted by a PDS, it is appended to the corresponding window partition within the SMV. The SMV therefore represents the continuously evolving accumulation of decentralized results. $SMVs$ serve as the effective base relation for $CMVs$, since they cannot directly reference client-local sources. SMV for query Q' represents the logical union of all $DMVs$ defined by Q across clients, where Q' is Q with an additional grouping by window:

$$SMV_{Q'} := \bigcup_i DMV_Q^{(i)} \tag{4}$$

The schema of $SMV_{Q'}$ matches that of each $DMV_Q^{(i)}$, preserving the system-defined metadata attributes.

Centralized tables and (materialized) views. We denote by S the centralized server; CT is a relation stored directly on S, independent of any client. CVs and $CMVs$ are time-varying relations defined by a query Q'', a relational algebra tree composed of operators from the set $\{\pi, \sigma, \bowtie, \gamma, \rho, \cup, \cap\}$ and may include any combination of these operators (joining multiple relations in the staging area is allowed). Let $R_D = \{R_1, \ldots, R_m\}$ denote the downstream relations in S or the PDSs, where each $R_j \in \{DMV, CMV, CV, CT\}$. Each query Q'' is evaluated pointwise in time as

$$CMV_{Q''}(\tau) = Q''(R_1(\tau), \ldots, R_m(\tau)). \tag{5}$$

Replicated materialized views. Each RMV is defined by a query Q'' on R_D, and thus varies over time as well. The contents of an RMV are identical across all clients C_i and do not evolve independently. [todo - can RMV have min agg? can RMV be extracted from the staging area?]

4.5 Incremental View Maintenance

The incremental algorithms apply uniformly to $DMVs$, $CMVs$, $RMVs$; for simplicity, we describe them here for $DMVs$. We use $MV_Q^{(i)}$ to denote a generic MV, instantiated as a DMV on C_i. Let $DT^{(i)}$ denote a decentralized table and $MV_Q^{(i)}$ the corresponding materialized view derived from query Q. We define the *delta* (Δ) of a relation R between two time instants τ_{k-1} and τ_k as

$$\Delta R(\tau_k) = R(\tau_k) \setminus R(\tau_{k-1}) \tag{6}$$

Each tuple in ΔR is annotated with a change type (`INSERT`, `DELETE`) and a timestamp t. For a given query Q, we define its incremental form IQ, which maps input deltas to the corresponding delta in the view:

$$\Delta MV_Q^{(i)} := IQ(\Delta R(\tau_k)^{(i)}) \tag{7}$$

where $\Delta DT^{(i)}$ denotes the collection of delta tables associated with the base relations used by Q. The updated view state is then obtained by merging the delta result into the previous materialized view:

$$MV_Q^{(i)}(\tau_k) := MV_Q^{(i)}(\tau_{k-1}) \oplus \Delta MV_Q^{(i)} \tag{8}$$

where $\oplus$ denotes a merge operator that applies the corresponding insertions and deletions to produce the new state.

Finally, the updated deltas of $MV_Q^{(i)}$ are propagated to SMV_Q. For clarity, we assume an append-only workload in this formalization, though the model generalizes naturally to deletions and updates.

In SIDRA, deletions are typically triggered when a client opts out, in which case all tuples associated with the corresponding `client_id` are removed from SMV_Q. In some cases, deletion from $CMVs$ is not possible, as data may have been aggregated beyond the point of attribution, and the batch of changes corresponding to stale windows may never reach the minimum aggregation threshold. Such aggregated results are retained, but they no longer

contain identifying or sensitive attributes and thus remain privacy-preserving.

4.6 View Constraints

TTL. `TTL` can be thought of as a user-defined watermark, defining the progress and behavior of events over time, as well as a triggering semantic that defines the lateness horizon to consider our data sufficiently complete. Of course, discarding data means that we may potentially never achieve true correctness or completeness; however, our aim is to enforce privacy while providing analytical value. We prioritize emitting "sufficiently good" data over potentially compromising either the materialization of timely results or user privacy, combining `TTL` and `REFRESH` to compute results, similar to the watermarks and triggers of Apache Beam [25].

The same notion can be applied to the constraints of correctness and completeness: the ability to ensure that query results always perfectly reflect the full dataset at any point in time is inherently unachievable. We therefore embrace the notion that results can be *sufficiently complete* rather than fully correct. Our IVM framework already partially mitigates incorrectness by allowing late arrivals to update previously materialized results, as long as they fall within the TTL and meet the minimum aggregation requirements. What is "sufficient" is application and context-dependent, and for this reason, SIDRA provides fine-grained per-window observability in terms of *completeness* and *responsiveness*.

Refresh. `REFRESH` controls when a materialized view is recomputed and its deltas emitted. Two refresh notions coexist: client-side, it establishes how often a PDS recomputes its DMV and sends its delta to the staging area. Server-side, it manifests how often SMV attempts to advance its CMV — factually "promoting" tuples from the staging area to the centralized component — as well as updating pipelines of CMVs.

Formally, let t_r denote a refresh time, ω the window assignment function, and $w_{curr} = \omega(t_r)$. We define the set of *active windows* in SMV_Q at t_r as:

$$W = \{w \mid w_{curr} - w \leq \texttt{TTL}\} \tag{9}$$

Each refresh processes all tuples in active windows, and the centralized view is updated by merging the staging contents based on their window and grouping key(s):

$$CMV_Q(\tau_k) := CMV_Q(\tau_{k-1}) \oplus \bigcup_{w \in W} \sigma_{\text{window}=w}(SMV_Q) \tag{10}$$

After each successful refresh, tuples belonging to expired windows are evicted from the staging view, shown in Eq. 11 (as well as the newly inserted tuples — omitted for simplicity):

$$SMV_Q(\tau_{k+1}) := SMV_Q(\tau_k) - \bigcup_{w \notin W} \sigma_{\text{window}=w}(SMV_Q) \tag{11}$$

`WINDOW` and `TTL` should be adjusted based on how often query results are needed, the available processing resources, and the value of out-of-scope tuples. For example, if the `MINIMUM AGGREGATION` requirement is not met, one could consider lowering its value or increasing the `TTL`. Conversely, `REFRESH` can be tuned on the basis of the tradeoff between freshness, performance, and privacy.

Minimum Aggregation. Given SMV_Q with dimension columns $\mathbf{p} = p_1, p_2, \ldots, p_k$, window, being refreshed at a window w_{curr} and minimum aggregation m_{agg}, we extend Eq. 10 filtering the tuples within the TTL and respecting m_{agg}. Let

$$CMV_Q(\tau_k) := \sigma_{\texttt{count(client_id)} \geq m_{\text{agg}} \wedge w_{curr} - \text{window} > \text{TTL}} \left(\gamma_{\mathbf{p}}(SMV_Q)\right) \tag{12}$$

Here, we omit the `DISTINCT` keyword due to set semantics. Lastly, tuples that do not meet the minimum aggregation with an expired TTL are discarded from CMV_Q. In other words, the only rows remaining in C_V after a refresh are those belonging to a non-expired window and having insufficient data points to meet the privacy requirements.

Currently, SIDRA applies a uniform minimum aggregation threshold per centralized view. However, future versions could support more fine-grained policies, allowing different columns or metrics to be governed by different aggregation levels.

Responsiveness *r(w)*. This metric, where $w \in \{1, \ldots, w_{curr}\}$ (per window) is useful to understand active clients, i. e. clients that have emitted a result in the current window. Our infrastructure offers the possibility to periodically remove inactive clients, and the responsiveness of the system can be used to tune how often this operation should be executed — for example, we can introduce a configuration parameter `minimum_responsiveness` calculating the (running) average for all windows and emitting a warning if the current value is lower than the setting. Let N be the total client count:

$$\text{clients_per_window} := \pi_{\text{window},\texttt{count(client_id)}} \left(\gamma_{\text{window}}(SMV_Q)\right) \tag{13}$$

$$r(\omega) = \frac{\text{clients_per_window}(w)}{N} \tag{14}$$

Completeness *c(w)*. This value indicates how much data for each window makes the minimum aggregation cut, and can be used to tune the refresh rate or the TTL — a higher value suggests a higher likelihood for enough data points to be collected. Before deleting the stale tuples when performing a `REFRESH` operation, we can infer their proportion w.r.t. the total number of rows in their window.

$$D := \pi_{\text{count}(*),\text{window}} \left(\sigma_{w_{curr} - \text{window} \leq \text{TTL}}(\gamma_{\text{window}}(SMV_Q))\right) \tag{15}$$

$$K := \pi_{\text{count}(*),\text{window}} \left(\gamma_{\text{window}}(CV_Q)\right) \tag{16}$$

$$c(w) = \frac{K(w)}{K(w) + D(w)} \tag{17}$$

Buffer Size *b(w)*. The buffer size represents the percentage of tuples still in the staging area for each window; for elapsed windows, it will be zero.

$$B := \pi_{\text{count}(*),\text{window}} \left(\gamma_{\text{window}}(SV_Q)\right) \tag{18}$$

$$b(w) = \frac{B(w)}{B(w) + D(w)} \tag{19}$$

Together, these semantics provide a formal model of eligibility and expiration. Because these rules are defined declaratively and embedded in the schema, they are also inspectable and auditable, helping end-users trust that privacy is enforced transparently, contributing to RQ1.3.

5 IMPLEMENTATION & EVALUATION

We implement SIDRA as two open-source extensions: `sidra_server`[5], a DuckDB extension that houses the compiler, refresh, and orchestrator logic, and `sidra_client`[6], a lightweight interface that adds a connectivity layer to PDSs running DuckDB. We choose DuckDB since it can be seamlessly integrated into mobile devices, browsers, laptops, and cloud platforms. Additionally, DuckDB is known for its low carbon footprint [10], which aligns with our goal of maximizing resource efficiency and minimizing reliance on cloud infrastructure. However, SIDRA can be adopted by any relational engine that supports SQL.

5.1 The SIDRA Compiler

To ensure compatibility with conventional relational systems, SIDRA SQL statements are compiled into standard SQL, targeting the centralized warehouse, the PDSs, and the staging area. We use a parser extension to recognize new keywords, remove them, and trigger the compiler to produce target-specific SQL, including metadata schema definitions and refresh plans.

For *DMVs*, our compiler performs the following: 1) creates the *MV* on the PDSs, 2) creates the *SMV* as a table with the same columns plus the additional metadata fields highlighted in Table 2, and 3) emits the refresh scripts for the pipeline. *CMVs* are defined in function of the aggregated data coming from the staging area, adding to its definition the metrics columns specified in Table 1. The SQL to update the observability columns is embedded into the refresh scripts.

IVM. Our IVM framework is also heavily based on SQL-to-SQL compilation. SIDRA builds on the open-source OpenIVM [13] package, that propagates changes from the base tables to ΔMV, and applies the latter to the *MV*. Any operation in the downstream sources (which can include tables and materialized views) is propagated to the respective delta table and assigned an action and timestamp through optimizer rules that intercept and copy insertions, deletions, and updates. We favor persisted table semantics [48] to simplify refresh propagation.

This IVM script is re-run every time a MV is refreshed. This "pure SQL" approach allows SIDRA to benefit from the mature relational ecosystem, including SQL engines such as SQLite (e. g., for PDSs), PostgreSQL (e. g., for managing data in the staging area), and DuckDB (for performing all IVM, and for analytics on *CMVs*). SIDRA enhances OpenIVM to generate SQL scripts that take into account the specified refresh intervals (i. e., the scripts only propagate tuples when refresh conditions are met, and enforce the privacy constraints described in §2).

Privacy Levels. [todo]

5.2 Differential Privacy

SIDRA can enforce differential privacy at two distinct layers: at the *DMV* level (local DP), where each PDS applies noise to its own grouped aggregates before sharing them, or at *SMV* (global DP), where noise is applied after obtaining grouped results from all PDSs, prior to publishing to *CMV*.

[5]https://github.com/ila/duckdb/tree/sidra-stable/extension/server
[6]https://github.com/ila/duckdb/tree/sidra-stable/extension/client

Further, we can apply noise based on composition-based DP [todo cite] or tree-based DP [todo cite]. To provide DP, SIDRA relies on the standard DP library Google DP, which enforces clipping, contribution bounding, and privacy-budget accounting [27]. This unified interface supports standard aggregates such as `SUM`, `COUNT`, and `MEAN`, allowing the same syntax to express both local and global DP queries. Eq. [todo] illustrates how the algebraic DP function in Eq. [todo] `dp_sum` can be translated in SQL.

```
dp_sum(column, epsilon, upper, lower)        (20)
```

Let N be the number of PDSs and let w_{curr} denote the active window. SIDRA guarantees bounded per-user contribution by construction: each PDS C_i contributes at most one clamped value to each group key g per refresh. Even if a PDS emits multiple tuples or arrives late, the staging area deterministically overwrites the previous value for (C_i, g). Thus, the global sensitivity of each aggregate is fixed by the clipping bounds $[\ell, u]$.

5.2.1 Composition DP. Composition-based DP treats each refresh as an independent release and splits the budget across all releases within the TTL horizon. In principle, both local and global DP satisfy formal definitions; in practice, staging-area DP provides higher utility under sparse or asynchronous participation, since it allows noise calibration over global data.

Global DP. Each *SMV* is assigned a privacy budget ε, managed by a trusted accountant that additionally establishes the clipping parameters $[\ell, u]$. The budget is divided across all releases triggered during the retention horizon: let $t_{TTL} = \frac{TTL}{window}$ be the number of windows covered by the TTL, and let $k = \frac{w}{r}$ be the number of refreshes per window. Under standard sequential composition, each refresh consumes

$$\varepsilon^{(refresh)} = \frac{\varepsilon}{t_{TTL} \cdot k} \tag{21}$$

Noise is then added independently at each refresh, calibrated to the global sensitivity Δf of the chosen aggregate (1 for COUNT, $u - \ell$ for SUM). At each refresh, the released noisy result for a group-window $G = (g, w_{curr})$ is

$$\widetilde{S}_g^{(w_{curr},t)} = S_g^{(w_{curr},t)} + \eta_t \qquad \eta_t \sim \mathrm{Lap}\left(\frac{\Delta f}{\varepsilon^{(refresh)}}\right) \tag{22}$$

Because each PDS contributes at most one clamped value per group and refresh, and because the staging area uses deterministic overwriting for late arrivals, the resulting mechanism satisfies $(\varepsilon, 0)$-DP over each TTL horizon for w_{curr} under sequential composition.

Local DP. In the local DP setting, each PDS C_i applies differential privacy to its own *DMV* outputs before sharing them. Each resulting noisy DMV satisfies $(\varepsilon_i, 0)$-DP locally, where ε_i corresponds to the PDS's full privacy budget for the refresh interval (as in Eq. 21). By construction, each PDS contributes at most one clipped value per group key g per refresh. At the *CMV* level, the locally noisy aggregates are summed across PDSs for each group-window $G = (g, w_{curr})$.

$$\widetilde{S}_g^{(w_{curr},t)} = \sum_{i=1}^{N} S_{g,i}^{(w_{curr},t)} + \sum_{i=1}^{N} \eta_{t,i}, \quad \eta_{t,i} \sim \mathrm{Lap}\left(\frac{\Delta f}{\varepsilon_i^{(refresh)}}\right) \tag{23}$$

Because each PDS's DP mechanism is independent, the resulting local group-level aggregates are formally private. However, the cumulative noise increases with N: each PDS randomizes its

own DMV contribution independently, and the SMV layer merely re-aggregates the N noisy reports. Under standard sequential composition, the aggregate satisfies at best $\varepsilon_{agg} = N \cdot \varepsilon$, which grows linearly with the population size. Consequently, while each PDS's output remains $(\varepsilon, 0)$-DP locally, the combined mechanism provides a strictly weaker privacy guarantee and injects substantially more noise than the global DP pipeline.

5.3 Tree DP

Tree-based DP arranges refreshes into a hierarchical structure and adds noise once per tree level, producing logarithmic privacy loss and substantially lower variance when refreshes are frequent.

Global DP. In the global setting, the SMV applies DP once per tree node: a tree of height $h = \lceil \log_2 k \rceil$ is built, and each node receives an equal share of the global privacy budget.

$$\varepsilon_{\text{node}} = \frac{\varepsilon}{h} \qquad \eta^{(j)} \sim \text{Lap}\left(\frac{\Delta f}{\varepsilon_{\text{node}}}\right) \tag{24}$$

A release at refresh time t is reconstructed as the sum of the logarithmically many noisy ancestors covering interval t:

$$\tilde{S}_g(w_{curr}, t) = \sum_{j \in \text{path}(t)} \left(S_g^{(j)} + \eta^{(j)}\right) \tag{25}$$

Since each individual contributes at most one clamped value per refresh and each refresh maps to $O(\log k)$ nodes, the mechanism satisfies $(\varepsilon, 0)$-DP over the full TTL horizon. Compared to composition DP, the per-release variance is reduced by a factor proportional to $\log k$.

Local DP. In the local setting, each PDS constructs its own DP tree over its per-refresh contributions on DMV. Each PDS C_i receives a local budget ε and splits it across the h tree levels (where h can be different than the global DP, as $DMVs$ can have a different refresh rate than $CMVs$). Each tree node for C_i adds noise calibrated to the local sensitivity (equal to the clipping range of the individual's contribution):

$$\eta_i^{(j)} \sim \text{Lap}\left(\frac{\Delta f}{\varepsilon_{\text{node}}}\right) \tag{26}$$

At the SMV layer, the final group-window release aggregates all PDSs reconstructed tree-summed values:

$$\tilde{S}_g(w_{curr}, t) = \sum_{i=1}^{N} \sum_{j \in \text{path}(t)} \left(S_{g,i}^{(j)} + \eta_i^{(j)}\right) \tag{27}$$

The mechanism remains locally private for each PDS. However, noise variance accumulates across both N users and $\log k$ nodes per path; thus, global utility is still substantially worse than in global tree DP, even though the logarithmic scaling reduces noise relative to naive local composition.

5.4 Evaluation

We evaluate how differential privacy behaves in SIDRA using a controlled synthetic benchmark that emulates a fitness tracker step-count aggregation pipeline. The system tracks step counts from N fitness trackers (PDSs) over a 24-hour period. Each tracker records hourly step measurements bounded in $[0, 4000]$ steps. We compare a standard centralized pipeline, where all the hourly measurements are sent to a central server, with SIDRA, firstly aggregating the partial sums for each PDS in a DMV, and then computes the global sums in a CMV. The DMV is refreshed every 4 hours (6 releases/day), while the CMV is refreshed every 12 hours (2 releases/day). For simplicity, we assume no late arrivals and a TTL of 1 window. We inject Laplace noise at different aggregation points and measure the resulting utility loss via mean absolute error (MAE). We compare six mechanisms organized into two families: raw, local, and global DP under sequential composition and binary tree aggregation. Each mechanism protects a different unit of contribution and uses different sensitivity and privacy-budget allocation.

Table 4: FIXME DP settings for the step-count benchmark. For tree-based mechanisms, h denotes tree height.

Strategy	Budget per Release/Node	Sensitivity Δf
Standard Composition (24 hourly releases)		
Raw DP	$\varepsilon/24$	4,000
Local DP	$\varepsilon/6$	$6 \times 4{,}000$
Global DP	$\varepsilon/2$	$2 \times 4{,}000$
Tree-Based Aggregation ($h = \lceil \log_2(24) \rceil = 5$ levels)		
Raw DP (tree)	$\varepsilon/5$	4,000
Local DP (tree)	$\varepsilon/5$	$24 \times 4{,}000$
Global DP (tree)	$\varepsilon/5$	$24 \times 4{,}000$

6 RELATED WORK

[todo add bigquery keywords] **Declarative Data Pipelines.** Many novel systems focus on declarative transformation pipelines: dbt (Data Build Tool) [19] is a popular framework that lets data architects transform and model their analytics data by writing modular SQL that gets compiled into MVs or tables inside a centralized data warehouse. Similarly, complementary efforts in platforms like Delta Live Tables [18] and Dynamic Tables [48] adapt MVs for pipeline orchestration on top of their analytical engine, unifying streaming and batching. Our system offers a similar declarative experience and extends it to a decentralized model, supporting hybrid aggregations, privacy constraints, and streaming primitives, enabling complex data flows while preserving privacy among PDS.

Incremental View Maintenance. IVM has been studied extensively: systems like DBToaster [4], Differential Dataflow [41], Noria [26], and DBSP [15] introduced increasingly sophisticated techniques for incrementally maintaining complex queries, ranging from joins, aggregates to recursive queries and different materialization strategies. While major database vendors (e. g. Oracle [14], Microsoft SQL Server [43], AWS Redshift [7], Google BigQuery [28]) have long supported MVs, their implementations remain geared toward accelerating static, batching queries. In contrast, recent IVM-based streaming systems like Materialize [40], RisingWave, and Feldera bring streaming-tailored latencies and declarative semantics to real-time workloads. Recent work further optimizes IVM for dynamic workloads, including edge-local privacy [29], and hybrid streaming-batch processing [1]. With SIDRA, we follow the

footsteps of these systems, but we focus on the privacy-aware integration of relational and stream processing paradigms to form more coarse-grained MV updates.

Decentralized Data Systems. The best-known attempt at creating a unified personal data store is Tim Berners Lee's SOLID project [46], but this does not envision central analytical queries. We believe that giving centralized query services access to analytics over whole fleets of personal data stores, via privacy-controlled MVs, will provide extra value that may help adoption of the concept of personal data stores, which until now has been lackluster.

MotherDuck [9], built around DuckDB, enables local-first analytics by distributing query execution between client and cloud. However, it does not support functionality of managing fleets of many clients with identical local tables (the PDSs in SIDRA) and does not support IVM at this moment, nor any specific functionality for ensuring privacy in query results. However, the hybrid query processing in MotherDuck could be used in SIDRA to communicate between decentralized clients and centralized servers using pure SQL, rather than using SIDRA's own runtime system for sending delta's over the wire.

Prior query-oriented decentralized infrastructures are mainly found in specific applications, such as real-time cellular network analytics systems exploiting geo-partitioning of input data [30]. Distributed and federated query processing are also the subject of extensive research. However, these always assume a free choice in data placement decisions [44, 47], rather than ascertaining that personal data is kept private and under the control of the end-user.

Trust frameworks [2, 23] and federated learning [16, 22] demonstrate the viability of decentralized privacy, though they lack relational query support. Recent work [54] proposes a user-centric, privacy-preserving, and verifiable ecosystem for personal data management, leveraging secure enclaves and federated learning to give users full control over heterogeneous personal information. The foundational work on secure computations by Yao [53] and privacy-preserving data mining by Agrawal and Srikant [3, 17, 50] further inform our approach. SIDRA complements this line of work in the database systems domain, leading to an approach where privacy is embedded in the architecture itself, generalizing the concept of decentralization.

NebulaStream [55] represents a complementary line of work, targeting adaptive query deployment across the edge through a unified streaming runtime. While NebulaStream focuses on efficient operator scheduling and resource-aware execution for IoT workloads, SIDRA addresses a different layer of the stack, with a declarative compilation model and privacy semantics for decentralized data aggregation. In principle, NebulaStream could serve as an engine in SIDRA.

Differential Privacy. A related research field to protect sensitive data is differential privacy [20, 21], the technique of adding noise to the data to obscure any individual's information while maintaining the statistical accuracy of the overall set. We think that our decentralized data architectures are a potential use case to build new forms of *decentralized* differential privacy, and note that research in this area tends to assume a central database. Beyond centralized DP, decentralized approaches like [16], incremental differentially private query processing [32, 51], private data federations [11], and edge-local DP [29] align with our architecture. Shuffle DP [36] and continual releases [52] offer mechanisms for privacy-preserving aggregations in streaming contexts. The mechanism design work by McSherry and Talwar [42] and practical implementations like Pinq [39] and PrivateSQL [33] provide solid foundations for differentially private DBMSs, while advances in federated DP [49] and SQL-based DP [31] offer system-level capabilities and insights. We also note that state-of-art differential privacy approaches within database systems such as Google DP [27] work best with aggregated data, matching our concept of aggregating MVs and integrating the relevant time dimension going hand-in-hand with our stream processing semantics: (i) *recency* of IVM updates will be part of our trade-offs against violating privacy constraints, and (ii) data architects may want to limit the lifetime of data in the MVs.

Notably, recent work on streaming differential privacy develops production-grade, centralized algorithms that provide formal DP guarantees for continual releases by combining private key selection, contribution bounding, and hierarchical aggregation [56]. SIDRA's approach is complementary: DP-SQLP's streaming-DP primitives are natural candidates to plug into SIDRAs client or staging steps to obtain formal DP guarantees, but doing so requires careful work to handle SIDRA's decentralization challenges (offline/asynchronous clients, contribution bounding, and composition accounting).

Similarly, SIDRA is compatible with the InkShrink framework [51], which formalizes sensitivity reduction for IVM under differential privacy. InkShrink's Transform and Shrink operators can be mapped applied to SIDRA's refreshes, providing an additional privacy layer. Moreover, SIDRA's staging area can be embedded within a MPC protocol, enabling InkShrink's joint guarantees for sensitive collaborative analytics.

7 CONCLUSION

SIDRA offers a privacy-friendly alternative to centralized data architectures through a three-tier design with streaming IVM and declarative privacy constraints. It enables data minimization, gives back user data ownership via PDSs, and lowers cloud infrastructure costs. SIDRA aligns with the core ideas of the recent "local first" movement, as it pushes application architectures towards providing data management functionality in a decentralized manner.

Whereas non-centralized architectures can be engineered without SIDRA, its advantage is its declarative approach based on the well-known and powerful SQL language. This strongly facilitates the development of decentralized data architectures and could make it a viable architectural alternative. The declarative nature of SIDRA SQL has the additional advantage that users can inspect and understand which data is transported to central servers, allowing for more insight and control over personal data. This work is based on the RDDA paper [todo cite] and grant [todo cite].

7.1 Future Work

Architectural Extensions. First, we plan to extend our infrastructure by making completeness and responsiveness metrics both dynamic and query-dependent. Currently, these are tracked only as metadata in centralized views: in the future, we aim to expose

them as explicit query-level constraints, such as requiring at least 80% client participation before emitting a result, or enforcing a minimum completeness threshold within each aggregation window.

Secure Query Processing. Strengthening privacy is a key research goal. First, protecting data at rest in PDSs requires more than local encryption; data must also be secure during use. Querying encrypted data remains an active research area [24], with potential optimizations like data skipping and vectorized encryption.

Second, securing the staging area is crucial to prevent privacy violations and build user trust. TEEs offer low overhead for analytics [12, 37] but have known vulnerabilities, while MPC [35] provides strong guarantees at higher performance cost [34]. We do not preemptively exclude either approach, as different use cases may have different requirements.

PAC Privacy. [todo]

REFERENCES

[1] Mahmoud Abo Khamis, Ahmet Kara, Dan Olteanu, and Dan Suciu. 2024. Insert-Only versus Insert-Delete in Dynamic Query Evaluation. *Proc. ACM Manag. Data* 2, 5, Article 219, 26 pages. https://doi.org/10.1145/3695837

[2] Will Abramson, Adam James Hall, Pavlos Papadopoulos, Nikolaos Pitropakis, and William J. Buchanan. 2020. A Distributed Trust Framework for Privacy-Preserving Machine Learning. In *Trust, Privacy and Security in Digital Business*, Stefanos Gritzalis, Edgar R. Weippl, Gabriele Kotsis, A. Min Tjoa, and Ismail Khalil (Eds.). Springer International Publishing, Cham, 205–220.

[3] Rakesh Agrawal and Ramakrishnan Srikant. 2000. Privacy-Preserving Data Mining. In *Proceedings of the 2000 ACM SIGMOD International Conference on Management of Data (SIGMOD '00)*. 439–450. https://doi.org/10.1145/342009.335438

[4] Yanif Ahmad, Oliver Kennedy, Christoph Koch, and Milos Nikolic. 2012. DBToaster: higher-order delta processing for dynamic, frequently fresh views. *Proc. VLDB Endow.* 5, 10 (jun 2012), 968979. https://doi.org/10.14778/2336664.2336670

[5] Tyler Akidau, Paul Barbier, Istvan Cseri, Fabian Hueske, Tyler Jones, Sasha Lionheart, Daniel Mills, Dzmitry Pauliukevich, Lukas Probst, Niklas Semmler, Dan Sotolongo, and Boyuan Zhang. 2023. What's the Difference? Incremental Processing with Change Queries in Snowflake. *Proc. ACM Manag. Data* 1, 2, Article 196 (jun 2023), 27 pages. https://doi.org/10.1145/3589776

[6] Tyler Akidau, Robert Bradshaw, Craig Chambers, Slava Chernyak, Rafael J Fernández-Moctezuma, Reuven Lax, Sam McVeety, Daniel Mills, Frances Perry, Eric Schmidt, et al. 2015. The dataflow model: a practical approach to balancing correctness, latency, and cost in massive-scale, unbounded, out-of-order data processing. *Proceedings of the VLDB Endowment* 8, 12 (2015), 1792–1803.

[7] Inc. Amazon Web Services. [n. d.]. Redshift Materialized Views. https://docs.aws.amazon.com/redshift/latest/dg/materialized-view-overview.html. Accessed: 2024-12-02.

[8] Arvind Arasu, Shivnath Babu, and Jennifer Widom. 2006. The CQL continuous query language: semantic foundations and query execution. *The VLDB Journal* 15, 2 (June 2006), 121142. https://doi.org/10.1007/s00778-004-0147-z

[9] R. J. Atwal, Peter Boncz, Ryan Boyd, Antony Courtney, Till Döhmen, Florian Gerlinghoff, Jeff Huang, Joseph Hwang, Raphael Hyde, Elena Felder, Jacob Lacouture, Yves LeMaout, Boaz Leskes, Yao Liu, Alex Monahan, Dan Perkins, Tino Tereshko, Jordan Tigani, Nick Ursa, Stephanie Wang, and Yannick Welsch. 2024. MotherDuck: DuckDB in the Cloud and in the Client. In *Proceedings of the 14th Conference on Innovative Data Systems Research (CIDR)*. Chaminade, USA. Creative Commons Attribution 4.0 International (CCBY4.0).

[10] Michail Bachras and Hans-Arno Jacobsen. 2025. Beyond Performance: Measuring the Environmental Impact of Analytical Databases. (2025). arXiv:2504.18980 [cs.DB] https://arxiv.org/abs/2504.18980

[11] Johes Bater, Xi He, William Ehrich, Ashwin Machanavajjhala, and Jennie Rogers. 2018. Shrinkwrap: efficient SQL query processing in differentially private data federations. *Proc. VLDB Endow.* 12, 3 (Nov. 2018), 307320. https://doi.org/10.14778/3291264.3291274

[12] Ilaria Battiston, Lotte Felius, Sam Ansmink, Laurens Kuiper, and Peter Boncz. 2024. DuckDB-SGX2: The Good, The Bad and The Ugly within Confidential Analytical Query Processing. In *Proceedings of the 20th International Workshop on Data Management on New Hardware* (Santiago, AA, Chile) *(DaMoN '24)*. Association for Computing Machinery, New York, NY, USA, Article 14, 5 pages. https://doi.org/10.1145/3662010.3663447

[13] Ilaria Battiston, Kriti Kathuria, and Peter Boncz. 2024. OpenIVM: a SQL-to-SQL Compiler for Incremental Computations. In *Companion of the 2024 International Conference on Management of Data* (Santiago AA, Chile) *(SIGMOD '24)*. Association for Computing Machinery, New York, NY, USA, 516519. https://doi.org/10.1145/3626246.3654743

[14] Randall Bello, Karl Dias, Alan Downing, James Jr, James Finnerty, William Norcott, Harry Sun, Andrew Witkowski, and Mohamed Ziauddin. 1998. Materialized Views in Oracle. 659–664.

[15] Mihai Budiu, Tej Chajed, Frank McSherry, Leonid Ryzhyk, and Val Tannen. 2023. DBSP: Automatic Incremental View Maintenance for Rich Query Languages. *Proc. VLDB Endow.* 16, 7 (mar 2023), 16011614. https://doi.org/10.14778/3587136.3587137

[16] Shuzhen Chen, Dongxiao Yu, Yifei Zou, Jiguo Yu, and Xiuzhen Cheng. 2022. Decentralized Wireless Federated Learning With Differential Privacy. *IEEE Transactions on Industrial Informatics* 18, 9 (2022), 6273–6282. https://doi.org/10.1109/TII.2022.3145010

[17] Chris Clifton, Murat Kantarcioglu, Jaideep Vaidya, Xiaodong Lin, and Michael Y. Zhu. 2002. Tools for privacy preserving distributed data mining. *SIGKDD Explor. Newsl.* 4, 2, 2834. https://doi.org/10.1145/772862.772867

[18] Inc. Databricks. [n. d.]. Delta Live Tables. https://www.databricks.com/product/delta-live-tables. Accessed: 2024-12-02.

[19] Inc. dbt Labs. [n. d.]. dbt. https://getdbt.com. Accessed: 2024-12-02.

[20] Cynthia Dwork. 2006. Differential Privacy. 4052 (2006), 1–12. https://doi.org/10.1007/11787006_1

[21] Cynthia Dwork. 2008. Differential Privacy: A Survey of Results. In *Theory and Applications of Models of Computation, 5th International Conference, TAMC 2008, Xi'an, China, April 25-29, 2008. Proceedings (Lecture Notes in Computer Science, Vol. 4978)*, Manindra Agrawal, Ding-Zhu Du, Zhenhua Duan, and Angsheng Li (Eds.). Springer, 1–19. https://doi.org/10.1007/978-3-540-79228-4_1

[22] Kairouz et. al. 2021. In *Advances and Open Problems in Federated Learning*.

[23] Alexandre Evfimievski, Johannes Gehrke, and Ramakrishnan Srikant. 2003. Limiting privacy breaches in privacy preserving data mining. In *Proceedings of the Twenty-Second ACM SIGMOD-SIGACT-SIGART Symposium on Principles of Database Systems* (San Diego, California) *(PODS '03)*. Association for Computing Machinery, New York, NY, USA, 211222. https://doi.org/10.1145/773153.773174

[24] Lotte Felius. 2024. Enhancing Security for Columnar Storage and Data Lakes. In *Proceedings of the VLDB Endowment*. 1–4.

[25] Apache Software Foundation. 2025. The Unified Apache Beam Model. https://beam.apache.org/ Accessed: 2025-09-15.

[26] Jon Gjengset, Malte Schwarzkopf, Jonathan Behrens, Lara Timbó Araújo, Martin Ek, Eddie Kohler, M Frans Kaashoek, and Robert Morris. 2018. Noria: dynamic, partially-stateful data-flow for high-performance web applications. In *13th USENIX Symposium on Operating Systems Design and Implementation (OSDI 18)*. 213–231.

[27] Google. 2025. Differential Privacy Library. https://github.com/google/differential-privacy Accessed: 2025-09-15.

[28] Inc. Google. [n. d.]. BigQuery Materialized Views. https://cloud.google.com/bigquery/docs/materialized-views-intro. Accessed: 2024-12-02.

[29] Yizhang He, Kai Wang, Wenjie Zhang, Xuemin Lin, Ying Zhang, and Wei Ni. 2025. Robust Privacy-Preserving Triangle Counting under Edge Local Differential Privacy. *Proc. ACM Manag. Data* 3, 3, Article 211, 26 pages. https://doi.org/10.1145/3725348

[30] Anand P. Iyer, Li Erran Li, and Ion Stoica. 2015. CellIQ : Real-Time Cellular Network Analytics at Scale. In *12th USENIX Symposium on Networked Systems Design and Implementation, NSDI 15, Oakland, CA, USA, May 4-6, 2015*. USENIX Association, 309–322. https://www.usenix.org/conference/nsdi15/technical-sessions/presentation/iyer

[31] Noah Johnson, Joseph P. Near, and Dawn Song. 2018. Towards practical differential privacy for SQL queries. *Proc. VLDB Endow.* 11, 5 (Oct. 2018), 526539. https://doi.org/10.1145/3177732.3177733

[32] Fumiyuki Kato, Tsubasa Takahashi, Shun Takagi, Yang Cao, Seng Pei Liew, and Masatoshi Yoshikawa. 2022. HDPView: differentially private materialized view for exploring high dimensional relational data. *Proc. VLDB Endow.* 15, 9 (May 2022), 17661778. https://doi.org/10.14778/3538598.3538601

[33] Ios Kotsogiannis, Yuchao Tao, Xi He, Maryam Fanaeepour, Ashwin Machanavajjhala, Michael Hay, and Gerome Miklau. 2019. PrivateSQL: a differentially private SQL query engine. *Proc. VLDB Endow.* 12, 11 (July 2019), 13711384. https://doi.org/10.14778/3342263.3342274

[34] John Liagouris, Vasiliki Kalavri, Muhammad Faisal, and Mayank Varia. 2021. Secrecy: Secure collaborative analytics on secret-shared data. *CoRR* abs/2102.01048 (2021). arXiv:2102.01048 https://arxiv.org/abs/2102.01048

[35] Yehuda Lindell. 2020. Secure Multiparty Computation. *Commun. ACM* 64, 1 (dec 2020), 8696. https://doi.org/10.1145/3387108

[36] Qiyao Luo, Jianzhe Yu, Wei Dong, Quanqing Xu, Chuanhui Yang, and Ke Yi. 2025. RM2: Answer Counting Queries Efficiently under Shuffle Differential Privacy. *Proc. ACM Manag. Data* 3, 3, Article 210, 24 pages. https://doi.org/10.1145/3725415

[37] Adrian Lutsch, Muhammad El-Hindi, Matthias Heinrich, Daniel Ritter, Zsolt István, and Carsten Binnig. 2025. Benchmarking Analytical Query Processing in Intel SGXv2. https://doi.org/10.48786/EDBT.2025.41

[38] Samuel Madden, Michael J. Franklin, Joseph M. Hellerstein, and Wei Hong. 2005. TinyDB: an acquisitional query processing system for sensor networks. *ACM Trans. Database Syst.* 30, 1 (2005), 122–173. https://doi.org/10.1145/1061318.1061322

[39] Frank McSherry. 2009. Privacy Integrated Queries. In *Proceedings of the 2009 ACM SIGMOD International Conference on Management of Data (SIGMOD)*. Association for Computing Machinery, Inc.

[40] Frank McSherry. 2022. Materialize: a platform for building scalable event based systems. In *Proceedings of the 16th ACM International Conference on Distributed and Event-Based Systems*. 3–3.

[41] Frank McSherry, Derek Gordon Murray, Rebecca Isaacs, and Michael Isard. 2013. Differential dataflow.. In *CIDR*.

[42] Frank McSherry and Kunal Talwar. 2007. Mechanism Design via Differential Privacy. In *Proceedings of the 48th Annual IEEE Symposium on Foundations of Computer Science (FOCS '07)*. 94–103. https://doi.org/10.1109/FOCS.2007.66

[43] Inc. Miscrosoft. [n. d.]. Azure Synapse Materialized Views. https://learn.microsoft.com/en-us/sql/t-sql/statements/create-materialized-view-as-select-transact-sql. Accessed: 2024-12-02.

[44] Hannes Mühleisen. 2012. *Architecture-independent distributed query processing*. Ph. D. Dissertation. Free University of Berlin. http://www.diss.fu-berlin.de/diss/receive/FUDISS_thesis_000000042056

[45] Andreas Pfitzmann and Marit Hansen. 2009. A terminology for talking about privacy by data minimization: Anonymity, Unlinkability, Undetectability, Unobservability, Pseudonymity, and Identity Management.

[46] Solid Project. 2025. Solid Project. https://solidproject.org/ Accessed: 2025-09-15.

[47] Mark Raasveldt and Hannes Mühleisen. 2018. MonetDBLite: An Embedded Analytical Database. *CoRR* abs/1805.08520 (2018). arXiv:1805.08520 http://arxiv.org/abs/1805.08520

[48] Daniel Sotolongo, Daniel Mills, Tyler Akidau, Anirudh Santhiar, Attila-Péter Tóth, Botong Huang, Boyuan Zhang, Igor Belianski, Ling Geng, Matt Uhlar, Nikhil Shah, Olivia Zhou, Saras Nowak, Sasha Lionheart, Vlad Lifliand, Wendy Grus, Yiwen Zhu, Ankur Sharma, Dzmitry Pauliukevich, Enrico Sartorello, Ilaria Battiston, Ivan Kalev, Lawrence Benson, Leon Papke, Niklas Semmler, Till Merker, and Yi Huang. 2025. Streaming Democratized: Ease Across the Latency Spectrum with Delayed View Semantics and Snowflake Dynamic Tables *(SIGMOD/PODS '25)*. Association for Computing Machinery, 622634. https://doi.org/10.1145/3722212.3724455

[49] Timothy Stevens, Christian Skalka, Christelle Vincent, John Ring, Samuel Clark, and Joseph Near. 2022. Efficient Differentially Private Secure Aggregation for Federated Learning via Hardness of Learning with Errors. In *31st USENIX Security Symposium (USENIX Security 22)*. USENIX Association, Boston, MA, 1379–1395. https://www.usenix.org/conference/usenixsecurity22/presentation/stevens

[50] Vassilios S. Verykios, Elisa Bertino, Igor Nai Fovino, Loredana Parasiliti Provenza, Yucel Saygin, and Yannis Theodoridis. 2004. State-of-the-art in privacy preserving data mining. *SIGMOD Rec.* 33, 1, 5057. https://doi.org/10.1145/974121.974131

[51] Chenghong Wang, Johes Bater, Kartik Nayak, and Ashwin Machanavajjhala. 2022. IncShrink: Architecting Efficient Outsourced Databases using Incremental MPC and Differential Privacy. In *Proceedings of the 2022 International Conference on Management of Data* (Philadelphia, PA, USA) *(SIGMOD '22)*. Association for Computing Machinery, New York, NY, USA, 818832. https://doi.org/10.1145/3514221.3526151

[52] Dongdong Xie, Pinghui Wang, Quanqing Xu, Chuanhui Yang, and Rundong Li. 2025. Efficient and Accurate Differentially Private Cardinality Continual Releases. *Proc. ACM Manag. Data* 3, 3, Article 151, 27 pages. https://doi.org/10.1145/3725288

[53] Andrew C. Yao. 1982. Protocols for secure computations. In *23rd Annual Symposium on Foundations of Computer Science (sfcs 1982)*. 160–164. https://doi.org/10.1109/SFCS.1982.38

[54] Osama Zafar, Mina Namazi, Yuqiao Xu, Youngjin Yoo, and Erman Ayday. 2025. A UserCentric, PrivacyPreserving, and Verifiable Ecosystem for Personal Data Management and Utilization. *arXiv preprint arXiv:cs.LG/157021* (July 2025). Case Western Reserve University.

[55] Steffen Zeuch, Ankit Chaudhary, Bonaventura Del Monte, Haralampos Gavriilidis, Dimitrios Giouroukis, Philipp M Grulich, Sebastian Bress, Jonas Traub, and Volker Markl. 2020. The NebulaStream Platform: Data and Application Management for the Internet of Things. In *Conference on Innovative Data Systems Research (CIDR)*. Amsterdam, Netherlands. https://hal.science/hal-02453998

[56] Bing Zhang, Vadym Doroshenko, Peter Kairouz, Thomas Steinke, Abhradeep Thakurta, Ziyin Ma, Eidan Cohen, Himani Apte, and Jodi Spacek. 2024. Differentially Private Stream Processing at Scale. *Proc. VLDB Endow.* 17, 12 (Aug. 2024), 41454158. https://doi.org/10.14778/3685800.3685833